

app.py

```
import streamlit as st
import pandas as pd
import sqlite3
import plotly.express as px
from pathlib import Path

st.set_page_config(
    page_title="Stock Market SQL Playground",
    page_icon="■",
    layout="wide"
)

# ----- Styling -----
st.markdown("""
<style>
.main-title {font-size: 2.4rem; font-weight: 800; margin-bottom: 0.2rem;}
.subtitle {color: #666; font-size: 1.05rem; margin-bottom: 1.5rem;}
.card {
    padding: 1rem 1.2rem;
    border-radius: 12px;
    border: 1px solid rgba(128,128,128,.25);
    background: rgba(128,128,128,.06);
}
</style>
""", unsafe_allow_html=True)

DATASETS = {
    "Bajaj Auto": ("Bajaj Auto.csv", "bajaj_auto"),
    "Eicher Motors": ("Eicher Motors.csv", "eicher_motors"),
    "Hero Motocorp": ("Hero Motocorp.csv", "hero_motocorp"),
    "Infosys": ("Infosys.csv", "infosys"),
}

ROOT = Path(__file__).parent

@st.cache_data
def load_data():
    data = {}
    for display_name, (filename, table_name) in DATASETS.items():
        path = ROOT / filename
        if path.exists():
            df = pd.read_csv(path)
            df.columns = [c.strip() for c in df.columns]
            data[display_name] = df
    return data

data = load_data()

def normalize_sql_name(name):
    return {
        "Date": "date",
        "Open Price": "open_price",
        "High Price": "high_price",
        "Low Price": "low_price",
        "Close Price": "close_price",
        "WAP": "wap",
        "No. of Shares": "no_of_shares",
        "No. of Trades": "no_of_trades",
        "Total Turnover (Rs.)": "total_turnover",
        "Deliverable Quantity": "deliverable_qty",
        "% Deli. Qty to Traded Qty": "pct_deli_qty",
        "Spread High-Low": "spread_high_low",
        "Spread Close-Open": "spread_close_open",
    }

def make_db(data):
    conn = sqlite3.connect(":memory:")
    for display_name, df in data.items():
        clean = df.rename(columns=normalize_sql_name).copy()
        if "date" in clean.columns:
            clean["date"] = pd.to_datetime(clean["date"], errors="coerce").dt.strftime("%Y-%m-%d")
        table = DATASETS[display_name][1]
        clean.to_sql(table, conn, index=False, if_exists="replace")
    return conn

conn = make_db(data)

# ----- Header -----
st.markdown('<div class="main-title">■ Stock Market Analysis – SQL Playground</div>', unsafe_allow_html=True)
st.markdown(
```

```

        '<div class="subtitle">Explore Bajaj Auto, Eicher Motors, Hero Motocorp and Infosys using SQL, tables and interactive cl
        unsafe_allow_html=True
    )

# ----- Sidebar -----
st.sidebar.header("■■■ Project Controls")
st.sidebar.success(f"{len(data)} stock datasets loaded")

for display_name, df in data.items():
    st.sidebar.write(f"***{display_name}* - {len(df):,} rows")

st.sidebar.markdown("---")
page = st.sidebar.radio(
    "Navigate",
    ["Dashboard", "SQL Playground", "Data Explorer", "Stock Charts", "Project Guide"]
)

# ----- Dashboard -----
if page == "Dashboard":
    st.subheader("Project Overview")

    cols = st.columns(4)
    for i, (name, df) in enumerate(data.items()):
        close_col = next((c for c in df.columns if c.lower() == "close price"), None)
        latest = df[close_col].iloc[-1] if close_col else None
        with cols[i]:
            st.metric(name, f"■{latest:,.2f}" if pd.notna(latest) else "N/A", f"{len(df):,} rows")

    st.markdown("---")

    stock = st.selectbox("Select stock", list(data.keys()))
    df = data[stock].copy()

    date_col = "Date"
    close_col = "Close Price"
    df[date_col] = pd.to_datetime(df[date_col], errors="coerce")
    df = df.sort_values(date_col)

    fig = px.line(
        df, x=date_col, y=close_col,
        title=f"{stock} - Closing Price History",
        labels={close_col: "Close Price (■)", date_col: "Date"}
    )
    fig.update_layout(hovermode="x unified")
    st.plotly_chart(fig, use_container_width=True)

    c1, c2 = st.columns(2)
    with c1:
        st.info("***Analysis focus:** historical prices, moving averages, trading signals, missing values and performance.")
    with c2:
        st.info("***Database:** SQLite is used inside Streamlit so the project runs without a separate database server.")

# ----- SQL Playground -----
elif page == "SQL Playground":
    st.subheader("■■■ SQL Playground")
    st.caption("Write SQL against the four loaded stock tables. SQL uses the normalized column names shown below.")

    schema_df = pd.DataFrame([
        ["bajaj_auto", "889", "date, open_price, high_price, low_price, close_price, wap, ..."],
        ["eicher_motors", "889", "date, open_price, high_price, low_price, close_price, wap, ..."],
        ["hero_motocorp", "889", "date, open_price, high_price, low_price, close_price, wap, ..."],
        ["infosys", "889", "date, open_price, high_price, low_price, close_price, wap, ..."],
    ], columns=["Table", "Rows", "Columns"])
    st.dataframe(schema_df, use_container_width=True, hide_index=True)

    examples = {
        "History - Bajaj Auto": """SELECT * FROM bajaj_auto ORDER BY date LIMIT 10;""",
        "Top 10 Eicher closing prices": """SELECT date, close_price
FROM eicher_motors
ORDER BY close_price DESC
LIMIT 10;""",
        "Infosys average close by year": """SELECT strftime('%Y', date) AS year,
ROUND(AVG(close_price), 2) AS avg_close
FROM infosys
GROUP BY year
ORDER BY year;""",
        "Missing deliverable quantities": """SELECT 'Bajaj Auto' AS stock, date
FROM bajaj_auto
WHERE deliverable_qty IS NULL
UNION ALL
SELECT 'Eicher Motors', date
FROM eicher_motors
WHERE deliverable_qty IS NULL
UNION ALL

```

```

SELECT 'Hero Motocorp', date
FROM hero_motocorp
WHERE deliverable_qty IS NULL
UNION ALL
SELECT 'Infosys', date
FROM infosys
WHERE deliverable_qty IS NULL;""",
    "Bajaj 20/50-day moving averages": """"WITH x AS (
        SELECT date, close_price,
            AVG(close_price) OVER (
                ORDER BY date ROWS BETWEEN 19 PRECEDING AND CURRENT ROW
            ) AS ma20,
            AVG(close_price) OVER (
                ORDER BY date ROWS BETWEEN 49 PRECEDING AND CURRENT ROW
            ) AS ma50
        FROM bajaj_auto
    )
    SELECT date, close_price,
        ROUND(ma20,2) AS ma20,
        ROUND(ma50,2) AS ma50
FROM x
WHERE ma20 IS NOT NULL AND ma50 IS NOT NULL
ORDER BY date;""",
    "Bajaj golden-cross signals": """"WITH ma AS (
        SELECT date, close_price,
            AVG(close_price) OVER (
                ORDER BY date ROWS BETWEEN 19 PRECEDING AND CURRENT ROW
            ) AS ma20,
            AVG(close_price) OVER (
                ORDER BY date ROWS BETWEEN 49 PRECEDING AND CURRENT ROW
            ) AS ma50
        FROM bajaj_auto
    ),
    lagged AS (
        SELECT *,
            LAG(ma20) OVER (ORDER BY date) AS prev_ma20,
            LAG(ma50) OVER (ORDER BY date) AS prev_ma50
        FROM ma
    )
    SELECT date, close_price,
        ROUND(ma20,2) AS ma20,
        ROUND(ma50,2) AS ma50,
        CASE
            WHEN ma20 > ma50 AND prev_ma20 <= prev_ma50 THEN 'Buy'
            WHEN ma20 < ma50 AND prev_ma20 >= prev_ma50 THEN 'Sell'
            ELSE 'Hold'
        END AS signal
FROM lagged
WHERE ma20 IS NOT NULL AND ma50 IS NOT NULL
ORDER BY date;""",
    "First vs last close – all four": """"WITH all_stocks AS (
        SELECT 'Bajaj Auto' stock, date, close_price FROM bajaj_auto
        UNION ALL SELECT 'Eicher Motors', date, close_price FROM eicher_motors
        UNION ALL SELECT 'Hero Motocorp', date, close_price FROM hero_motocorp
        UNION ALL SELECT 'Infosys', date, close_price FROM infosys
    )
    SELECT stock,
        MIN(date) AS first_date,
        MAX(date) AS last_date,
        ROUND((SELECT a.close_price
            FROM all_stocks a
            WHERE a.stock=s.stock AND a.date=MIN(s.date)),2) AS first_close,
        ROUND((SELECT a.close_price
            FROM all_stocks a
            WHERE a.stock=s.stock AND a.date=MAX(s.date)),2) AS last_close
FROM all_stocks s
GROUP BY stock;""",
    }

example = st.selectbox("Load an example query", list(examples.keys()))
default_sql = examples[example]

sql = st.text_area("SQL Query", value=default_sql, height=260)

if st.button("■ Run SQL", type="primary"):
    try:
        result = pd.read_sql_query(sql, conn)
        st.success(f"Query executed successfully – {len(result):,} rows returned.")
        st.dataframe(result, use_container_width=True, hide_index=True)

        csv = result.to_csv(index=False).encode("utf-8")
        st.download_button(
            "■ Download result as CSV",
            data=csv,

```

```

        file_name="sql_result.csv",
        mime="text/csv"
    )

    numeric = result.select_dtypes(include="number").columns.tolist()
    if len(result) > 1 and len(numeric) >= 1:
        st.markdown("### ■ Automatic Visualization")
        x_options = [c for c in result.columns if c not in numeric]
        x_col = st.selectbox("X-axis", x_options if x_options else result.columns.tolist())
        y_col = st.selectbox("Y-axis", numeric)
        fig = px.line(result, x=x_col, y=y_col, markers=True)
        st.plotly_chart(fig, use_container_width=True)
    except Exception as e:
        st.error(f"SQL Error: {e}")

# ----- Data Explorer -----
elif page == "Data Explorer":
    st.subheader("■ Data Explorer")
    stock = st.selectbox("Choose dataset", list(data.keys()))
    df = data[stock]

    st.write(f"**{stock}** – {len(df):,} rows × {len(df.columns)} columns")
    st.dataframe(df, use_container_width=True, height=520)

    st.markdown("### Column Information")
    info = pd.DataFrame({
        "Column": df.columns,
        "Data Type": [str(df[c].dtype) for c in df.columns],
        "Missing Values": [int(df[c].isna().sum()) for c in df.columns],
    })
    st.dataframe(info, use_container_width=True, hide_index=True)

# ----- Stock Charts -----
elif page == "Stock Charts":
    st.subheader("■ Interactive Stock Charts")

    stock = st.selectbox("Choose stock", list(data.keys()))
    df = data[stock].copy()
    df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
    df = df.sort_values("Date")

    tab1, tab2, tab3 = st.tabs(["Closing Price", "Moving Averages", "Trading Volume"])

    with tab1:
        fig = px.line(df, x="Date", y="Close Price", title=f"{stock} Closing Price")
        st.plotly_chart(fig, use_container_width=True)

    with tab2:
        df["MA20"] = df["Close Price"].rolling(20).mean()
        df["MA50"] = df["Close Price"].rolling(50).mean()
        ma = df.dropna(subset=["MA20", "MA50"])
        fig = px.line(ma, x="Date", y=["Close Price", "MA20", "MA50"],
                      title=f"{stock} – 20/50 Day Moving Averages")
        st.plotly_chart(fig, use_container_width=True)

    with tab3:
        fig = px.bar(df, x="Date", y="Total Turnover (Rs.)",
                    title=f"{stock} Total Turnover")
        st.plotly_chart(fig, use_container_width=True)

# ----- Guide -----
else:
    st.subheader("■ Project Guide")
    st.markdown("""
    ### Objective
    Analyze historical stock-market data using SQL and present the findings through an interactive Streamlit application.

    ### Main Tasks
    1. Explore stock history and top closing prices.
    2. Calculate yearly average closing prices.
    3. Identify missing values.
    4. Calculate 20-day and 50-day moving averages.
    5. Detect golden-cross Buy/Sell signals.
    6. Compare first and last closing prices.
    7. Identify large day-over-day movements.
    8. Apply the specified Infosys bonus adjustment where required.
    9. Convert SQL results into clear charts and project insights.

    ### Tables
    - `bajaj_auto`
    - `eicher_motors`
    - `hero_motocorp`
    - `infosys`

    ### Run the Project
    """)

```

```
```bash
pip install -r requirements.txt
streamlit run app.py
```
```

The application uses SQLite locally, so no MySQL server is required to run the Streamlit playground.
""")

```
st.markdown("---")
st.caption("Stock Market Analysis SQL Playground • College Project")
```